

EXERCICE3

Compte rendu Exercice 3 : analyse des dépendances avec Trivy

Méthode utilisée

J'ai commencé par cloner le dépôt cible :

```
git clone https://github.com/veracode/verademo ./target
```

J'ai ensuite lancé Trivy en mode filesystem afin qu'il analyse les fichiers de dépendances présents dans le projet.

```
docker compose run --rm trivy
```

La commande exécutée est équivalente à :

```
trivy fs --scanners vuln --offline-scan --format json --output  
reports/trivy-fs-report.json /repo
```

L'option `--offline-scan` a été utilisée pour éviter les appels vers les dépôts Maven distants. Sans cette option, Trivy peut rencontrer une erreur de type `429 Too Many Requests`, car Maven Central limite le nombre de requêtes.

Résultat du scan

Dans le terminal, Trivy affiche principalement des logs d'exécution :

```
INFO    [vuln] Vulnerability scanning is enabled  
WARN    [pom] Dependency version cannot be determined. Child dependencies will not  
be found.  
INFO    Number of language-specific files          num=1  
INFO    [pom] Detecting vulnerabilities...
```

Le message d'avertissement signifie que certaines dépendances enfants ne pourront pas être résolues entièrement. Trivy poursuit l'analyse du fichier `pom.xml`.

Le rapport complet est écrit dans le fichier suivant :

```
reports/trivy-fs-report.json
```

Après analyse du rapport JSON, le résultat obtenu est :

```
total: 18
CRITICAL: 3
HIGH: 10
MEDIUM: 5
```

Les vulnérabilités critiques identifiées sont :

```
CVE-2022-22965    spring-boot-starter-web 2.3.1.RELEASE
CVE-2015-7501     commons-collections4 4.0
CVE-2016-1000031  commons-fileupload 1.3.2
```

Analyse

La vulnérabilité la plus marquante est `CVE-2022-22965`, aussi connue sous le nom de Spring4Shell. Elle concerne Spring et peut permettre une exécution de code à distance dans certains contextes. Son score CVSS est de 9.8, ce qui la classe parmi les vulnérabilités critiques.

Les deux autres vulnérabilités critiques concernent `commons-collections4` et `commons-fileupload`. Ce sont des bibliothèques Java connues, souvent présentes dans des applications anciennes. Leur présence dans des versions vulnérables montre l'intérêt d'une analyse régulière des dépendances.

Réponses aux questions

Combien de vulnérabilités sont détectées ?

Trivy détecte 18 vulnérabilités au total : 3 critiques, 10 hautes et 5 moyennes.

Quelle est la vulnérabilité la plus critique ?

La vulnérabilité la plus critique est `CVE-2022-22965`, liée à Spring. Elle est particulièrement importante car elle peut permettre une exécution de code à distance.

Quelle dépendance est concernée ?

La dépendance concernée est `spring-boot-starter-web` en version `2.3.1.RELEASE`.

Quelle version corrige le problème ?

La correction consiste à mettre à jour Spring Boot vers `2.5.12`, `2.6.6` ou une version plus récente.

Plan de correction

Je commencerais par corriger les vulnérabilités critiques. La priorité serait de mettre à jour Spring Boot, puis `commons-collections4` et `commons-fileupload`. Ensuite, je traiterais les vulnérabilités hautes, en vérifiant à chaque étape que l'application reste fonctionnelle.

Après les mises à jour, il faudrait relancer Trivy pour confirmer que les CVE ont bien disparu. Dans un contexte réel, ce type de contrôle devrait être intégré à la CI/CD, avec un outil comme Renovate ou

Dependabot pour faciliter le suivi des dépendances.